DESDE EL MÓDULO 1 · S1 · S3 · S6 · S11 · FUNCIÓN DE PÉRDIDA · CLASIFICACIÓN · ÁRBOLES

Impureza Gini

(y su prima, la entropía)

`en el código · criterion='gini' · 'entropy'`

Para qué sirve. Medir qué tan mezclado está un grupo de filas: 0 si todas son de la misma clase, máximo si están a mitades. Un árbol elige el corte que más la reduce.

LA FÓRMULA

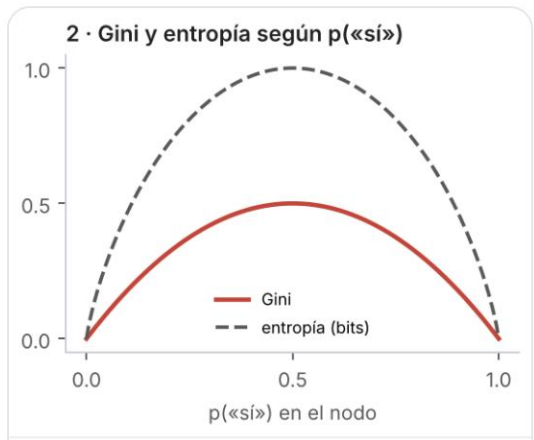
$G = 1 - \Sigma_k\ p_k^2$ · Gini del corte = $(n_{izq}/n)\cdot G_{izq} + (n_{der}/n)\cdot G_{der}$ · ganancia = $G_{padre} - G_{corte}$

Léela con las cuatro viñetas de abajo: la figura muestra lo que la ecuación dice.

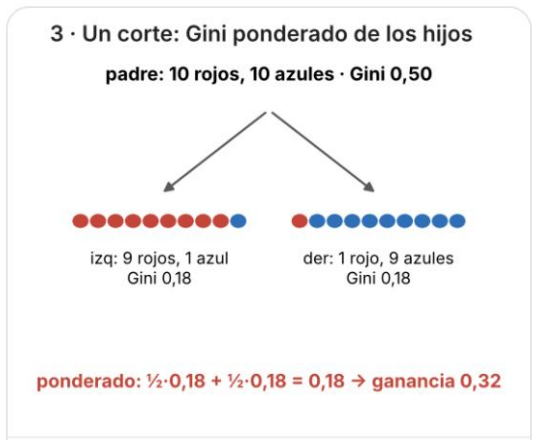
1 CÓMO FUNCIONA — LA HISTORIA EN CUATRO PASOS



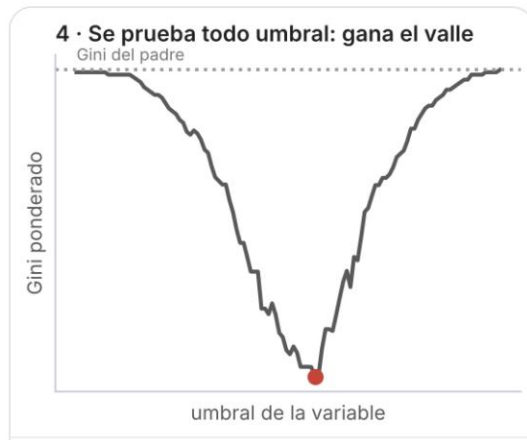
1. Un nodo es un grupo de filas. Si todas son rojas, Gini = 0 (puro). Siete rojas y tres azules: $1 - 0{,}7^2 - 0{,}3^2 = 0{,}42$. Mitad y mitad: 0,50, el máximo con dos clases.



2. Gini es una parábola en p: cero en los extremos y máxima en 0,5. La entropía ($-\Sigma\ p \log_2 p$) tiene la misma forma y casi siempre elige el mismo corte.



3. Un corte parte el padre en dos hijos. Se calcula el Gini de cada hijo y se pondera por cuántas filas recibió. La ganancia es lo que bajó respecto al padre.



4. El árbol repite ese cálculo para cada variable y cada umbral posible, y se queda con el valle más profundo. Es búsqueda exhaustiva: no hay gradiente.

2 ASÍ SE CALCULA EN PYTHON · Y QUÉ SIGNIFICA CADA TÉRMINO

```python
import numpy as np

def gini(y):                                          # 1
    p = np.bincount(y) / len(y)                       # 2
    return 1 - np.sum(p ** 2)

def gini_corte(y_izq, y_der):                         # 3
    n = len(y_izq) + len(y_der)
    return (len(y_izq)/n * gini(y_izq)
          + len(y_der)/n * gini(y_der))

ganancia = gini(y) - gini_corte(y[mask], y[~mask])    # 4
DecisionTreeClassifier(criterion='gini')              # 5
```

1. **`gini(y)`**
   Recibe las clases de las filas de un nodo y devuelve un número entre 0 (puro) y 1 − 1/K (todo mezclado). Con 2 clases, máximo 0,5.
2. **`p = proporciones`**
   p_k es la fracción de filas de cada clase en el nodo. Es lo único que Gini mira: no le importa el orden ni los valores de x.
3. **`gini_corte`**
   Promedio ponderado de los hijos. Un hijo con 2 filas pura casi no pesa: por eso min_samples_leaf evita hojas diminutas «perfectas».
4. **`ganancia`**
   Cuánto bajó la impureza. El árbol elige el (variable, umbral) con mayor ganancia y repite en cada hijo.
5. **`criterion`**
   En sklearn cambias la fórmula con una palabra. 'entropy' (o 'log_loss') usa la entropía; en la práctica, mismos árboles.

3 QUÉ LA HACE ÚTIL, QUÉ VIGILAR Y DÓNDE APARECE

VENTAJAS

- ✔ Barata: solo cuenta proporciones, sin logaritmos.
- ✔ Se calcula por nodo: permite probar miles de cortes rápido.
- ✔ Funciona igual con 2 o con 20 clases.

CUIDADO CON

- ! Premia variables con muchos valores distintos (fechas, IDs, `direccion`).
- ! No sabe de costos: trata igual un falso «sí» y un falso «no».
- ! No se puede derivar: por eso el árbol busca, no desciende.

QUIÉN LA USA EN EL CURSO

ÁRBOL (S1, S3)	Elige cada corte minimizando el Gini ponderado; la profundidad decide cuántas veces.
BOSQUE (S6)	El mismo Gini en cada árbol, pero cada corte solo ve √p variables candidatas.
BOOSTING (S11)	Sus árboles pequeños parten por una ganancia parecida, pero sobre los residuales del log-loss, no sobre las clases.
IMPORTANCIAS	`feature_importances_` = cuánta reducción de Gini aportó cada variable, sumada en el bosque.

EN EL CURSO Figura `arbol_como_elige_el_corte` (S1). Con `max_depth=None` el árbol lleva el Gini a 0 en 20 403 hojas: memorizó (S3).

DESDE EL MÓDULO 2 · S4 · S7 · S11 · FUNCIÓN DE PÉRDIDA · REGRESIÓN · CLUSTERING

Suma de cuadrados

(error cuadrático medio, MSE)

`en el código · mean_squared_error · r2_score`

Para qué sirve. Sumar los errores al cuadrado para que una recta (o un centroide) quede lo más cerca posible de todos los puntos a la vez, castigando más los errores grandes.

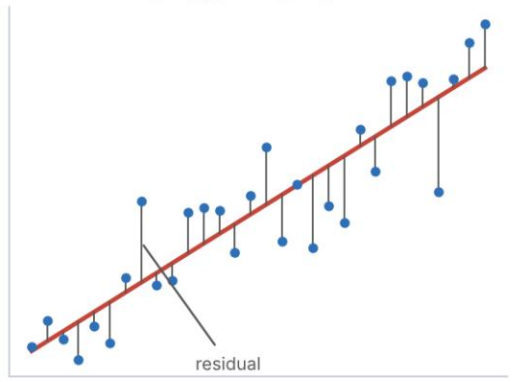
LA FÓRMULA

$$L(\beta) = \Sigma_i (y_i - \hat{y}_i)^2 \cdot MSE = L/n \cdot R^2 = 1 - L / \Sigma(y_i - \bar{y})^2$$

Léela con las cuatro viñetas de abajo: la figura muestra lo que la ecuación dice.

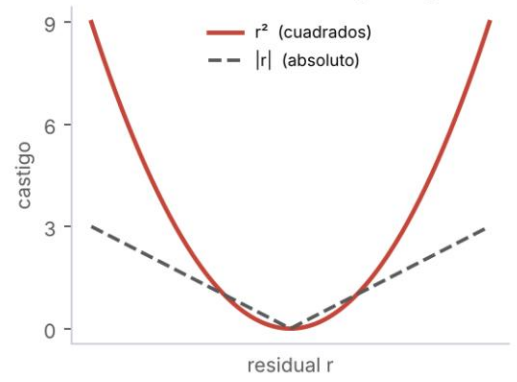
1 CÓMO FUNCIONA — LA HISTORIA EN CUATRO PASOS

1 · Residual = y − ŷ, punto por punto



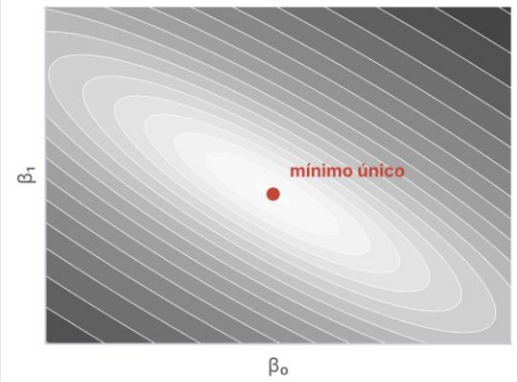
1 Para cada fila, el residual es la distancia vertical entre lo que pasó (y) y lo que el modelo dijo (ŷ). Positivo si el modelo se quedó corto, negativo si se pasó.

2 · Elevar al cuadrado castiga los grandes



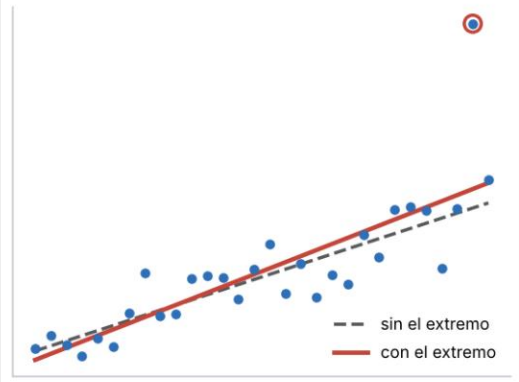
2 Elevar al cuadrado hace dos cosas: borra el signo (un error de −3 y uno de +3 pesan igual) y castiga desproporcionadamente los errores grandes: $3^2 = 9$, pero $1^2 = 1$.

3 · Sumados sobre (β_0, β_1): un tazón



3 Sumados sobre todas las filas, y vistos como función de los parámetros (β_0, β_1), forman un tazón convexo: un solo fondo. Por eso hay fórmula cerrada.

4 · Un valor extremo mueve la recta



4 El precio del cuadrado: un valor extremo arrastra la recta hacia él. Si eso duele, la alternativa es el error absoluto (MAE), que no se puede derivar tan fácil.

2 ASÍ SE CALCULA EN PYTHON · Y QUÉ SIGNIFICA CADA TÉRMINO

```python
import numpy as np

residuales = y - y_pred                                  # 1
sce = np.sum(residuales ** 2)                            # 2
mse = np.mean(residuales ** 2)                           # 3
rmse = np.sqrt(mse)                                      # 4
r2 = 1 - sce / np.sum((y - y.mean()) ** 2)               # 5

from sklearn.metrics import mean_squared_error, r2_score
mean_squared_error(y_test, y_pred), r2_score(y_test, y_pred)
```

1 **residuales**
El vector de errores, uno por fila. Su gráfica contra ŷ es el primer diagnóstico: debe verse como ruido sin forma.

2 **sce (suma)**
Lo que la regresión lineal minimiza al pie de la letra. Crece con n, así que no sirve para comparar datasets de distinto tamaño.

3 **mse (media)**
La misma cantidad dividida por n: comparable entre conjuntos. Sus unidades son las de y al cuadrado (pesos², puntos²).

4 **rmse (raíz)**
Vuelve a las unidades de y: «el modelo se equivoca en promedio ±4 600 USD». Es la que se reporta al negocio.

5 **R^2**
Fracción de la varianza explicada: 0 = igual que la media, 1 = perfecto. Negativo en test = peor que el baseline tonto.

3 QUÉ LA HACE ÚTIL, QUÉ VIGILAR Y DÓNDE APARECE

VENTAJAS

- ✔ Derivable y convexa: fórmula cerrada y descenso de gradiente funcionan.
- ✔ Tiene interpretación estadística (máxima verosimilitud con ruido normal).
- ✔ RMSE y R^2 se explican en una frase.

CUIDADO CON

- ! Sensible a valores extremos: un punto raro mueve todo.
- ! $R^2 = 1$ no es un logro, es una alarma de fuga (S4).
- ! MSE de train siempre baja al agregar variables; solo el de test cuenta.

QUIÉN LA USA EN EL CURSO

REGRESIÓN LINEAL (S4)	La minimiza exacta con $(X^TX)^{-1}X^Ty$ o paso a paso con descenso de gradiente.
RIDGE Y LASSO (S4)	La misma suma más una penalización sobre los β.
K-MEANS (S7)	Es su inercia: suma de distancias² de cada punto a su centroide. Mismo cuadrado, otra distancia.
BOOSTING EN REGRESIÓN (S11)	Cada árbol nuevo ajusta los residuales, que son el gradiente de esta pérdida.

EN EL CURSO `fg.superficie_perdida()` y `fg.recta_residuales()` en S4 (insurance). El R^2 = 1,000 de Saber 11 es la trampa del módulo.

DESDE EL MÓDULO 2 · S4 · S5 · S11 · TÉRMINO DE PÉRDIDA · REGRESIÓN · CLASIFICACIÓN · BOOSTING

Penalización L2 y L1

(regularización: el freno de los coeficientes)

`en el código · alpha · C · l2_regularization`

Para qué sirve. Sumar a la pérdida un costo por coeficientes grandes, para que el modelo solo use un β grande cuando los datos lo justifiquen de verdad. Es la herramienta general contra el overfitting.

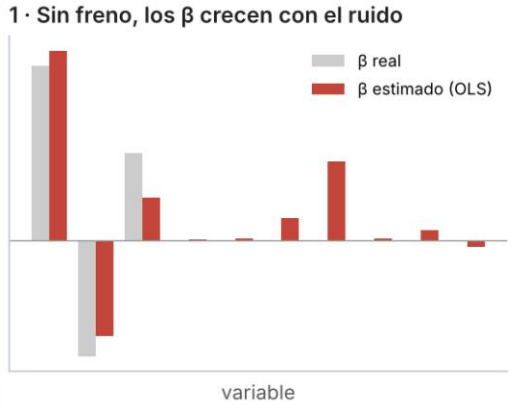
LA FÓRMULA

$L_{total} = L_{datos} + \alpha \cdot \Sigma\beta_j^2$ (L2, Ridge) ·
$L_{total} = L_{datos} + \alpha \cdot \Sigma|\beta_j|$ (L1, Lasso)

Léela con las cuatro viñetas de abajo: la figura muestra lo que la ecuación dice.

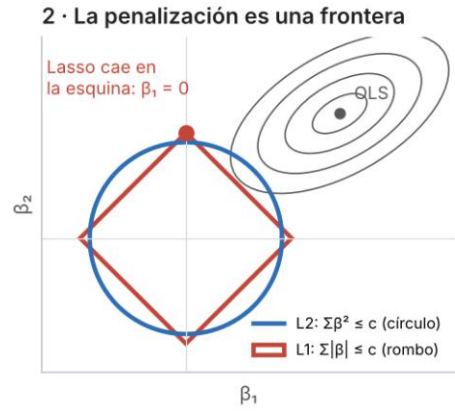
1 CÓMO FUNCIONA — LA HISTORIA EN CUATRO PASOS

1 · Sin freno, los β crecen con el ruido



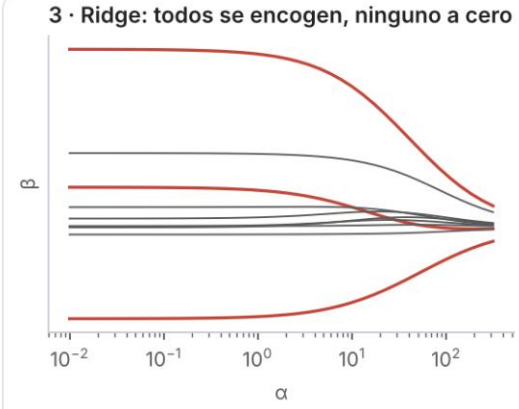
1. Con muchas variables y pocas filas, OLS asigna coeficientes a variables que en realidad valen 0: está ajustando el ruido. En test, esos β sobran.

2 · La penalización es una frontera



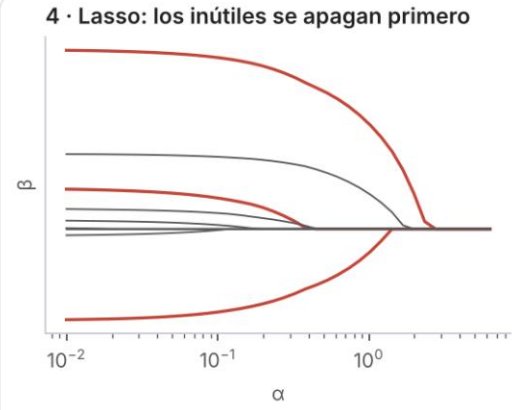
2. Penalizar equivale a obligar a los β a vivir dentro de una frontera. L2 es un círculo: suave, nadie llega exacto a 0. L1 es un rombo con esquinas sobre los ejes: la solución cae en una esquina y algún β se vuelve 0.

3 · Ridge: todos se encogen, ninguno a cero



3. Al subir α (más freno), Ridge encoge todos los coeficientes hacia 0 de forma gradual. Las variables reales (rojas) resisten más tiempo que las de ruido.

4 · Lasso: los inútiles se apagan primero



4. Lasso apaga primero las variables inútiles y conserva las reales hasta α grande. Es selección de variables automática; con α enorme, apaga todo.

2 ASÍ SE CALCULA EN PYTHON · Y QUÉ SIGNIFICA CADA TÉRMINO

```python
# la pérdida que Ridge minimiza, a mano
def perdida_ridge(beta, X, y, alpha):
    residual = y - X @ beta
    return (np.sum(residual ** 2)                    # 1
            + alpha * np.sum(beta ** 2))             # 2

Ridge(alpha=10)                                      # 3
Lasso(alpha=0.5)        # alpha * sum(|beta|)
LogisticRegression(C=0.1)                            # 4
HistGradientBoostingClassifier(l2_regularization=1.0)  # 5
```

1. **`L_datos`**
   La pérdida de siempre: cuadrados en regresión, log-loss en clasificación. La penalización se suma, no la reemplaza.
2. **`α · Σβ² (o Σ|β|)`**
   El freno. α = 0 → modelo libre. α → ∞ → todos los β a 0 (predice la media). β_0 nunca se penaliza.
3. **`alpha`**
   Nombre en Ridge y Lasso. Se elige con validación cruzada (`RidgeCV` , `LassoCV` o GridSearch), nunca mirando el test.
4. **`C = 1/α`**
   En LogisticRegression y SVM el mismo freno se escribe al revés: C pequeño = mucho freno. Trampa clásica de lectura.
5. **`l2_regularization`**
   En gradient boosting penaliza los valores de las hojas de cada árbol. Mismo principio: hojas más tímidas.

3 QUÉ LA HACE ÚTIL, QUÉ VIGILAR Y DÓNDE APARECE

VENTAJAS

- ✔ Una sola perilla controla el overfitting de forma continua.
- ✔ L1 entrega modelos cortos y explicables.
- ✔ Hace estables problemas con variables correlacionadas o p > n.

CUIDADO CON

- ! Sin escalar las x, α castiga más a las variables con unidades pequeñas.
- ! Los β encogidos ya no son el «efecto puro» de cada variable.
- ! α elegido en train = optimista; se valida con CV (S10).

QUIÉN LA USA EN EL CURSO

RIDGE / LASSO (S4)	Cuadrados + L2 o L1 sobre las decenas de dummies de Saber 11.
LOGÍSTICA (S5)	Log-loss + L2 por defecto, con C = 1/α. Sin C grande, los β no se disparan con variables separables.
BOOSTING (S11)	`l2_regularization` sobre las hojas; junto con `learning_rate` y la parada temprana, es su freno.
REDES NEURONALES (FUERA DEL CURSO)	El mismo término se llama weight decay.

EN EL CURSO `fg.regularizacion_alpha()` en S4. El contraste Ridge/Lasso sobre Saber 11 muestra qué variables sobreviven.

DESDE EL MÓDULO 2 · S5 · S6 · S11 · FUNCIÓN DE PÉRDIDA · CLASIFICACIÓN CON PROBABILIDADES

Log-loss

(entropía cruzada · −log-verosimilitud)

`en el código · log_loss · predict_proba`

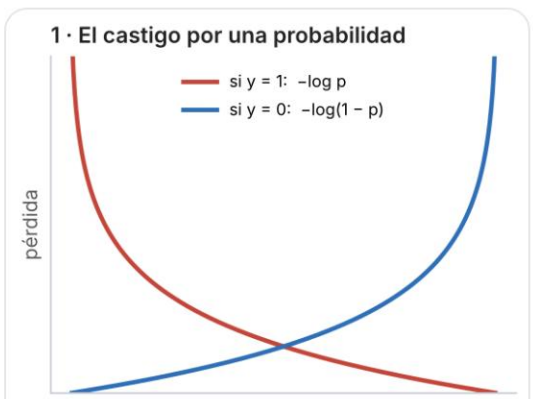
Para qué sirve. Castigar a un clasificador según la probabilidad que le dio a lo que realmente pasó: poco si acertó con confianza, muchísimo si se equivocó con confianza.

LA FÓRMULA

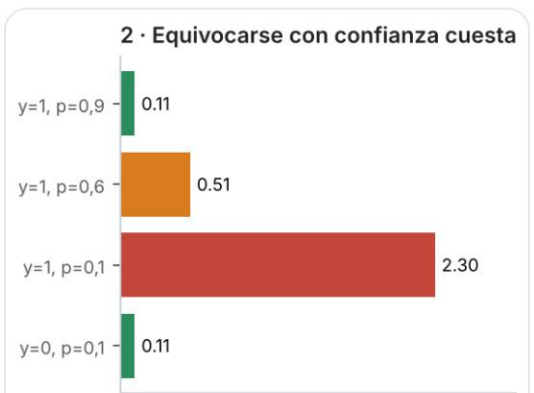
$$L = -(1/n)\ \Sigma_i\ [\ y_i \cdot \log p_i + (1 - y_i) \cdot \log(1 - p_i)\ ] \cdot p_i = \sigma(z_i)$$

Léela con las cuatro viñetas de abajo: la figura muestra lo que la ecuación dice.

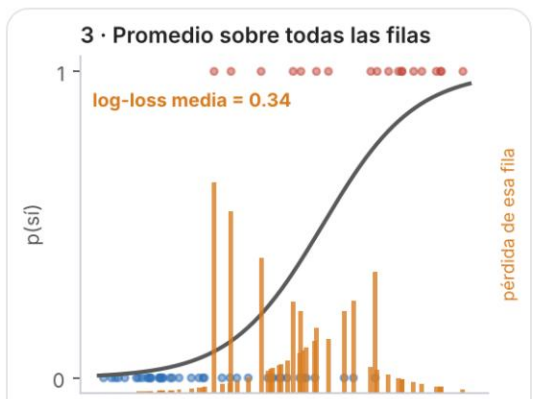
1 CÓMO FUNCIONA — LA HISTORIA EN CUATRO PASOS



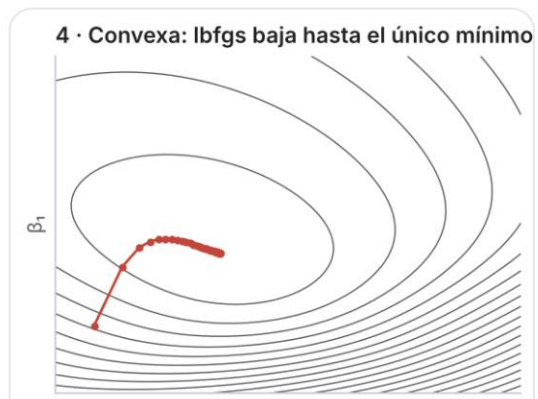
1 Para una fila con y = 1, la pérdida es −log p. Si p = 0,9, cuesta 0,11; si p = 0,5, cuesta 0,69; si p = 0,01, cuesta 4,6. Para y = 0 es la curva espejo con 1 − p.



2 Compara: acertar con 0,9 cuesta 0,11; equivocarse con 0,1 cuesta 2,30. Veinte veces más. La log-loss premia la calibración, no solo la etiqueta.



3 La pérdida total es el promedio sobre todas las filas. Las barras naranjas muestran quién aporta: los puntos del lado equivocado de la sigmoide.



4 Vista sobre los β es convexa (un solo mínimo) pero sin fórmula cerrada: lbfgs baja iterando. Por eso LogisticRegression tiene max_iter y la lineal no.

2 ASÍ SE CALCULA EN PYTHON · Y QUÉ SIGNIFICA CADA TÉRMINO

```python
import numpy as np

p = modelo.predict_proba(X)[:, 1]                    # 1
eps = 1e-15
p = np.clip(p, eps, 1 - eps)                         # 2
perdida_fila = -(y * np.log(p)
                 + (1 - y) * np.log(1 - p))          # 3
log_loss = perdida_fila.mean()                       # 4

from sklearn.metrics import log_loss
log_loss(y_test, modelo.predict_proba(X_test))       # 5
```

1 **`p = predict_proba`**
La pérdida se calcula sobre probabilidades, nunca sobre la clase predicha. Un modelo que solo da 0/1 no tiene log-loss (sería infinita).

2 **`clip`**
log(0) = −∞. Se recorta p a $[10^{-15}, 1 - 10^{-15}]$ para que una sola fila no reviente el promedio.

3 **`perdida_fila`**
Solo uno de los dos términos sobrevive: si y = 1 queda −log p; si y = 0 queda −log(1 − p).

4 **`mean`**
Promedio, no suma: comparable entre conjuntos. Baseline: predecir siempre la proporción de «sí» (en Bank: 0,36).

5 **`log_loss (sklearn)`**
Recibe la matriz completa de `predict_proba` (dos columnas) y hace el clip por ti.

3 QUÉ LA HACE ÚTIL, QUÉ VIGILAR Y DÓNDE APARECE

VENTAJAS

- ✔ Derivable y convexa: el descenso de gradiente llega al óptimo.
- ✔ Mide calibración: distingue un 0,51 de un 0,99.
- ✔ Funciona con K clases (entropía cruzada multiclase).

CUIDADO CON

- ! Un error confiado domina el promedio: revisa los extremos.
- ! No se lee en unidades de negocio: para eso, matriz de confusión y ganancia (S5).
- ! Optimizarla no fija el umbral: eso sigue siendo decisión tuya.

QUIÉN LA USA EN EL CURSO

LOGÍSTICA (S5)	La minimiza con lbfgs; la `ConvergenceWarning` es esta pérdida que no terminó de bajar.
BOOSTING EN CLASIFICACIÓN (S11)	Cada árbol nuevo apunta al gradiente de esta pérdida: y − p, el «residual» de una probabilidad.
ÁRBOL CON CRITERION='LOG_LOSS'	Misma fórmula usada como impureza del nodo (= entropía).
COMPARAR MODELOS (S10)	`scoring='neg_log_loss'` en CV cuando importan las probabilidades, no solo el AUC.

EN EL CURSO `fg.sigmoide_y_frontera()` en S5. El accuracy 0,894 con recall 0,18 tenía una log-loss mediocre: la pérdida sí veía el problema.

DESDE EL MÓDULO 3 · S5 · S7 · S8 · FUNCIÓN DE PÉRDIDA · VECINOS · CLUSTERING · PCA

Distancia e inercia

(lo que miden k-NN, K-means y DBSCAN)

`en el código · euclidean · inertia_ · eps`

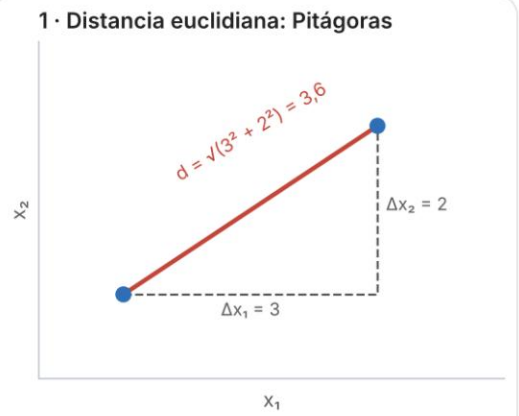
Para qué sirve. Convertir «qué tan parecidas son dos filas» en un número. Los algoritmos de vecinos y de clusters no tienen y: la distancia es toda su definición de verdad.

LA FÓRMULA

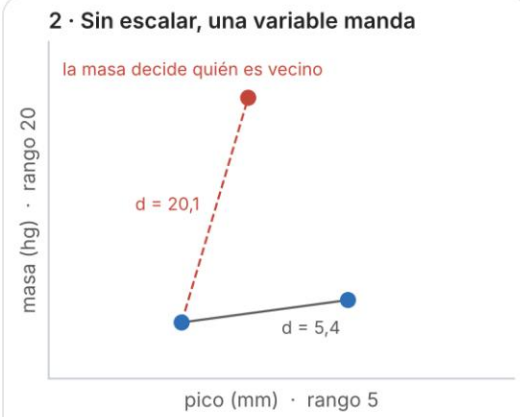
$d(x, x') = \sqrt{\Sigma_j (x_j - x'_j)^2}$ · $\text{inercia} = \Sigma_k \Sigma_{i \in C_k} \|x_i - \mu_k\|^2$ · PCA maximiza Σ varianza proyectada

Léela con las cuatro viñetas de abajo: la figura muestra lo que la ecuación dice.

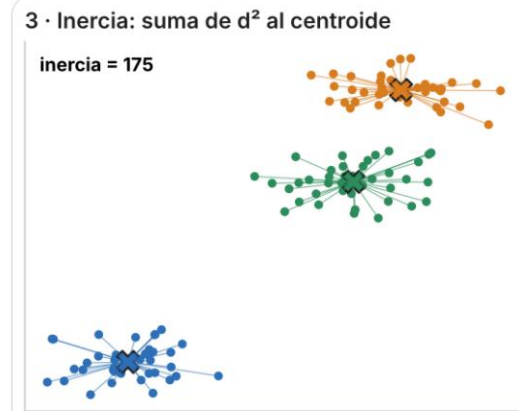
1 CÓMO FUNCIONA — LA HISTORIA EN CUATRO PASOS



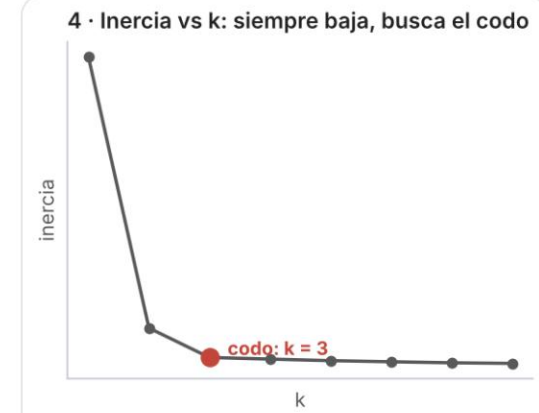
1. La distancia euclidiana es Pitágoras en p dimensiones: raíz de la suma de diferencias al cuadrado, variable por variable.



2. Cada variable aporta su diferencia al cuadrado en sus propias unidades. Si una se mide en gramos y otra en milímetros, la de rango grande decide sola quién es vecino. Escalar es obligatorio.



3. K-means suma las distancias² de cada punto a su centroide: la inercia. Es la suma de cuadrados de la regresión, con el centroide en el papel de ŷ.



4. La inercia siempre baja al subir k (con k = n es 0). Por eso no sirve para elegir k por sí sola: se busca el codo, o se usa silhouette, o el sentido del negocio.

2 ASÍ SE CALCULA EN PYTHON · Y QUÉ SIGNIFICA CADA TÉRMINO

```python
import numpy as np
from sklearn.preprocessing import StandardScaler

X_esc = StandardScaler().fit_transform(X)                    # 1
d = np.sqrt(np.sum((X_esc[0] - X_esc[1]) ** 2))              # 2

km = KMeans(n_clusters=4).fit(X_esc)
inercia = sum(np.sum((X_esc[km.labels_ == k]
                      - km.cluster_centers_[k]) ** 2)
              for k in range(4))                             # 3
km.inertia_                                                  # 4
DBSCAN(eps=0.5)   # radio en unidades escaladas              # 5
```

1. **StandardScaler**
Resta la media y divide por la desviación: cada variable pasa a tener rango comparable. Sin esto, nada de lo que sigue significa algo.

2. **d (euclidiana)**
Una distancia por par de filas. `metric='manhattan'` suma |diferencias| sin cuadrado: menos sensible a una variable extrema.

3. **inercia a mano**
Suma por cluster de las distancias² al centroide. Minimizarla es exactamente lo que K-means hace alternando asignar y recalcular.

4. **inertia_**
sklearn la guarda tras el fit. Se grafica contra k para el codo; con `n_init=10` se queda con el arranque de menor inercia.

5. **eps**
En DBSCAN la distancia no se minimiza: es un umbral. ε = 0,5 significa «medio desvío estándar» si escalaste.

3 QUÉ LA HACE ÚTIL, QUÉ VIGILAR Y DÓNDE APARECE

VENTAJAS

- ✔ Una sola idea sirve a k-NN, K-means, DBSCAN, PCA y t-SNE.
- ✔ Sin supuestos sobre la forma de la relación.
- ✔ La inercia se calcula en una línea y se explica con una figura.

CUIDADO CON

- ! Depende por completo de la escala y de las unidades.
- ! En muchas dimensiones todo queda igual de lejos de todo.
- ! Variables categóricas no tienen distancia natural: one-hot las distorsiona.

QUIÉN LA USA EN EL CURSO

K-NN (S5)	No minimiza nada: ordena por distancia y deja votar a los k más cercanos. Recall 0,20 → 0,03 sin escalar.
K-MEANS (S7)	Minimiza la inercia por alternancia. Mínimo local: `n_init` repite el arranque.
DBSCAN (S7)	Usa la distancia como umbral (ε) para decidir núcleo, borde y ruido.
PCA (S8)	Maximiza la varianza (distancia² a la media) conservada al proyectar: la misma cantidad, al revés.

EN EL CURSO `fg.knn_vecinos()` (S5) y `fg.kmeans_iteraciones()` (S7). Trampa: K-means sin escalar agrupa Penguins por gramos.

DESDE EL MÓDULO 2 · S4 · S5 · S8 · S11 · OPTIMIZADOR · CUALQUIER PÉRDIDA DERIVABLE

Descenso de gradiente

(y la tasa de aprendizaje η)

`en el código · learning_rate · max_iter · tol`

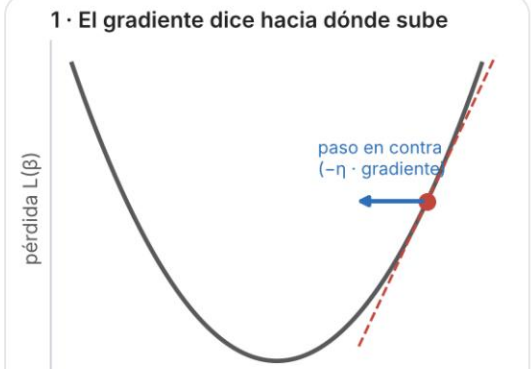
Para qué sirve. Encontrar el mínimo de una pérdida sin fórmula cerrada: mirar la pendiente donde estás, dar un paso pequeño cuesta abajo, repetir. Es el motor de la logística, t-SNE, boosting y las redes neuronales.

LA FÓRMULA

$\beta \leftarrow \beta - \eta \cdot \nabla L(\beta)$ · ∇L = pendiente de la pérdida respecto a cada parámetro · parar cuando $|\Delta L| < \text{tol}$

Léela con las cuatro viñetas de abajo: la figura muestra lo que la ecuación dice.

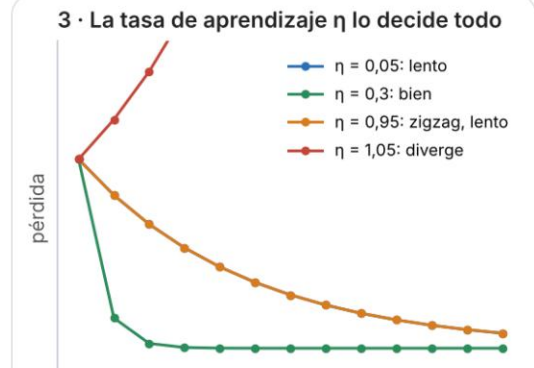
1 CÓMO FUNCIONA — LA HISTORIA EN CUATRO PASOS



1. El gradiente ∇L es la pendiente de la pérdida en el punto actual: dice hacia dónde sube. El algoritmo da el paso en sentido contrario, con tamaño η veces la pendiente.



2. Lejos del mínimo la pendiente es grande y los pasos son largos; cerca, la pendiente se aplana y los pasos se acortan solos. Converge sin que nadie le diga dónde está el fondo.



3. η (tasa de aprendizaje) es la perilla del optimizador: muy pequeña y no llega en max_iter; adecuada y baja limpio; grande y zigzaguea; demasiado grande y la pérdida sube (diverge).



4. Si la pérdida no es convexa (t-SNE, boosting, redes), hay varios valles. Dónde arrancas (semilla) decide en cuál caes. Por eso t-SNE cambia con random_state.

2 ASÍ SE CALCULA EN PYTHON · Y QUÉ SIGNIFICA CADA TÉRMINO

```python
import numpy as np
beta = np.zeros(X.shape[1]); eta = 0.01                    # 1
for i in range(1000):                                      # 2
    y_pred = X @ beta
    gradiente = -2 * X.T @ (y - y_pred) / len(y)           # 3
    beta = beta - eta * gradiente                          # 4
    if np.linalg.norm(gradiente) < 1e-6: break             # 5

LogisticRegression(max_iter=1000, tol=1e-4)    # lbfgs
HistGradientBoostingClassifier(learning_rate=0.05)
TSNE(learning_rate='auto')
```

1. **`eta (η)`**
   Tamaño del paso. Es la perilla que se enseña en S4 con la fila de 4 figuras. Regla: la más grande que no haga subir la pérdida.
2. **`max_iter`**
   Tope de pasos. Si se alcanza sin converger, sklearn lanza `ConvergenceWarning`: la pérdida aún bajaba.
3. **`gradiente`**
   Para cuadrados es $-2X^T(y - \hat{y})$: la derivada de la pérdida respecto a cada β. Para log-loss es $X^T(p - y)$. Cada pérdida tiene el suyo.
4. **`beta − eta·gradiente`**
   La única línea que importa: moverse un poquito en contra de la pendiente. Todo lo demás es contabilidad.
5. **`tol`**
   Tolerancia: cuando el gradiente (o el cambio en la pérdida) es más chico que esto, se declara convergido.

3 QUÉ LA HACE ÚTIL, QUÉ VIGILAR Y DÓNDE APARECE

VENTAJAS

- ✔ Sirve para cualquier pérdida derivable, con millones de parámetros.
- ✔ Escala a datos que no caben en memoria (versión estocástica, por lotes).
- ✔ Es el mismo algoritmo de la logística, boosting, t-SNE y deep learning.

CUIDADO CON

- ! η mal elegida: o no llega o diverge.
- ! En pérdidas no convexas cae en mínimos locales.
- ! Necesita variables escaladas: con rangos distintos, el tazón es un cañón y zigzaguea.

QUIÉN LO USA EN EL CURSO

REGRESIÓN LINEAL (S4)	Implementado a mano por el estudiante; llega al mismo punto que la fórmula cerrada. Existe para cuando no hay fórmula.
LOGÍSTICA (S5)	lbfgs: un primo que además estima la curvatura y elige η solo. Sin fórmula cerrada, es obligatorio.
T-SNE (S8)	Mueve posiciones 2-D para bajar la divergencia KL. No convexa: cada semilla, un mapa.
GRADIENT BOOSTING (S11)	El mismo paso, pero cada «β» es un árbol nuevo que apunta al gradiente del anterior. η = `learning_rate`.

EN EL CURSO `fg.descenso_gradiente_paso_a_paso()` y `fg.learning_rate_efecto()` (S4). Lasso usa descenso por coordenadas: una β a la vez.

MÓDULOS 1–4 · S1 → S11 · HILO TRANSVERSAL · 13 ALGORITMOS Y TÉCNICAS

Mapa: qué minimiza cada algoritmo y cómo

El hilo «Dentro del .fit()» en una sola página

Para qué sirve. Cada algoritmo del curso es una función de pérdida más un procedimiento para minimizarla. Esta tabla junta las fichas de los 13 para verlas de una vez: la misma pregunta, cuatro renglones, trece respuestas.

1 LA TABLA: CUATRO PREGUNTAS, TRECE RESPUESTAS

ALGORITMO	TIPO	QUÉ MINIMIZA (O MAXIMIZA)	MOVIENDO QUÉ	CÓMO LO BUSCA	PERILLA DEL OPTIMIZADOR
Árbol de decisión Módulo 1 · S1 · S3	Supervisado	Impureza Gini de cada corte Gini	Variable y umbral de cada pregunta	Búsqueda exhaustiva y voraz Búsqueda	Ninguna (max_depth es del modelo)
Regresión lineal Módulo 2 · S4	Supervisado	Suma de residuales² Cuadrados	$\beta_0, \beta_1, \ldots, \beta_p$	Fórmula cerrada o descenso de gradiente Cerrada	η, iteraciones (si usas gradiente)
Ridge · Lasso Módulo 2 · S4	Supervisado	Cuadrados + $\alpha\cdot\Sigma\beta^2$ / $\alpha\cdot\Sigma\lvert\beta\rvert$ Cuadrados L1/L2	$\beta_0, \beta_1, \ldots, \beta_p$	Ridge: cerrada · Lasso: por coordenadas Cerrada	tol, max_iter (α es del modelo)
Regresión logística Módulo 2 · S5	Supervisado	Log-loss (+ L2 vía C) Log-loss L1/L2	$\beta_0, \beta_1, \ldots, \beta_p$	Iterativo: lbfgs (gradiente + curvatura) Gradiente	max_iter, tol
k-NN Módulo 2 · S5	Supervisado	Nada: no hay pérdida Distancia	Nada	No optimiza: memoriza y compara al predecir Sin optimización	— (k es del modelo)
Bosque aleatorio Módulo 2 · S6	Supervisado	Gini en cada árbol Gini	Los cortes de cada árbol	Muchas búsquedas voraces sobre bootstrap Búsqueda	Ninguna
K-means Módulo 3 · S7	No supervisado	Inercia: Σ distancias² al centroide Distancia	Asignaciones y centroides	Alternancia asignar → recalcular (mínimo local) Alternancia	n_init, max_iter
DBSCAN Módulo 3 · S7	No supervisado	Nada: usa la distancia como umbral Distancia	Nada	Recorre vecindades y encadena núcleos Sin optimización	— (ε, min_samples son del modelo)
PCA Módulo 3 · S8	No supervisado	Maximiza varianza proyectada Distancia	Direcciones (autovectores)	Descomposición de la covarianza: cerrada Cerrada	—
t-SNE Módulo 3 · S8	No supervisado	Divergencia KL entre vecindades KL	Posiciones 2-D de cada punto	Descenso de gradiente no convexo Gradiente	learning_rate, perplexity, max_iter
Gradient boosting Módulo 4 · S11	Supervisado	Log-loss o cuadrados (+ L2 en hojas) Log-loss L1/L2	Árboles pequeños en fila	Gradiente en el espacio de árboles Gradiente	learning_rate, nº de árboles, tamaño
Validación cruzada Módulo 4 · S10	Evaluación	— (mide, no entrena)	—	Repite fit/score K veces	K, barajado, estratificación
Búsqueda de hiperparámetros Módulo 4 · S11	Optimización	Métrica de validación cruzada	Hiperparámetros del modelo	Grilla · azar · bayesiana (TPE): caja negra Caja negra	Tamaño de malla, n_iter, n_trials

2 LEYENDA Y LAS TRES IDEAS QUE HAY QUE LLEVARSE

FUNCIONES DE PÉRDIDA

- **Gini** — **Impureza Gini** · Ficha P1
- **Cuadrados** — **Suma de cuadrados** · Ficha P2
- **L1/L2** — **Penalización L1/L2** · Ficha P3
- **Log-loss** — **Log-loss** · Ficha P4
- **Distancia** — **Distancia · inercia · varianza** · Ficha P5
- **KL** — **Divergencia KL** · Solo t-SNE; se explica en S8

OPTIMIZADORES

- **Cerrada** — **Fórmula cerrada / descomposición** · salta al mínimo en un paso
- **Gradiente** — **Descenso de gradiente (y lbfgs)** · baja por la pendiente, paso a paso
- **Búsqueda** — **Búsqueda exhaustiva y voraz** · prueba todo, se queda con lo mejor, no vuelve
- **Alternancia** — **Alternancia** · fija una cosa, optimiza la otra, repite
- **Caja negra** — **Caja negra (grilla, azar, bayesiana)** · sin gradiente: evalúa configuraciones
- **Sin optimización** — **Sin optimización** · memoriza o aplica un umbral

LO QUE HAY QUE RECORDAR

1. Hiperparámetros del modelo (profundidad, k, α, ε) cambian QUÉ se minimiza o con qué forma; los del optimizador (η, max_iter, tol, n_init) cambian CÓMO se busca.
2. Convexa (cuadrados, log-loss, Ridge) = un solo valle, cualquier arranque llega. No convexa (K-means, t-SNE, boosting) = la semilla importa.
3. Elegir hiperparámetros también es optimizar: otra pérdida (la de validación), sin gradiente. Optuna aprende dónde buscar; el ganador siempre luce mejor en CV que en test.

EN EL CURSO Sección 6.5 del documento maestro. La ficha de cuatro renglones aparece en la celda A′ de cada algoritmo; esta página las junta.